

LAPORAN PROJECT BIG CHALLENGE
BIG CHALLENGE STURKTUR DATA DAN ALGORITMA



Dosen Pengampu:

Prof. Dr. Taufik Fuadi Abidin, M.Tech.

Dr. Ir. Irvanizam, S.Si., M.Sc., IPM., ASEAN Eng.

Disusun oleh:

Muhammad Aqil Mubarak (250810701100003)

Virzi Mayhand Syahputra (250810701100077)

JURUSAN INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA
BANDA ACEH
2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan Project Big Challenge pada mata kuliah Struktur Data dan Algoritma ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai bentuk pertanggungjawaban akademik sekaligus sarana penerapan konsep dan logika struktur data menggunakan bahasa C.

Laporan ini membahas perancangan dan implementasi program yang mampu membaca serta memproses dataset *Bag of Words*, melakukan *parsing* data dokumen teks, menghitung serta mengakumulasi frekuensi kemunculan setiap kata, hingga mengurutkan data dalam jumlah besar menggunakan berbagai algoritma pengurutan (seperti *Insertion Sort*, *Quick Sort*, dan *Heap Sort*). Melalui project ini, penulis diharapkan dapat memahami penerapan struktur data yang efisien, analisis kompleksitas algoritma, serta pengelolaan proses baca-tulis (*input/output*) pada file teks.

Penulis mengucapkan terima kasih kepada Prof. Dr. Taufik Fuadi Abidin, M.Tech dan Dr. Ir. Irvanizam, S.Si., M.Sc., IPM., ASEAN Eng. selaku dosen pengasuh mata kuliah Struktur Data dan Algoritma atas arahan dan bimbingan yang telah diberikan. Ucapan terima kasih juga disampaikan kepada rekan-rekan yang telah memberikan dukungan selama pengerjaan project ini.

Penulis menyadari bahwa laporan ini masih memiliki keterbatasan, sehingga kritik dan saran yang membangun sangat diharapkan. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dalam memahami penerapan struktur data, efisiensi algoritma pengurutan, dan pengolahan data berskala besar menggunakan bahasa pemrograman C.

Banda Aceh, 15 Juni 2026
Penyusun

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PERMASALAHAN.....	1
1.1 Masalah.....	1
BAB II METODE STRUKTUR DATA DAN ALGORITMA.....	2
2.1 Desain Program.....	2
2.2 Flowchart Program.....	3
2.3 Struktur Data	3
2.4 Proses Parsing dan Pembacaan Data.....	4
2.5 Analisis Algoritma	6
2.5.1 Insertion Sort.....	6
2.5.2 Quick Sort	7
2.5.3 Heap Sort	7
2.6 Penyimpanan hasil ke dalam file text.....	8
BAB III HASIL PROGRAM	9
3.1 Tampilan Awal dan Pemuatan Dataset	9
3.2 Tampilan Menu Utama	9
3.3 Output Proses Pengurutan dan Pembuatan File	9
3.3.1 Menu pilihan pertama	9
3.3.2 Menu pilihan kedua.....	10
3.3.3 Menu pilihan ketiga	10
3.4 Isi File Output	10
3.4.1 File output Insertion Sort	10
3.4.2 File output Quick Sort.....	11
3.4.3 File output Heap Sort.....	11
3.4 Output K Kata Teratas di Layar Monitor	11
3.5 Pengecekan memory leaks	12
BAB IV KESIMPULAN.....	13
LAMPIRAN.....	14

BAB I

PERMASALAHAN

1.1 Masalah

Dataset *Bag of Words* yang digunakan dalam tugas ini berasal dari *Machine Learning Repository* (UCI) dan berisi representasi teks dari berbagai sumber (seperti PubMed, NYTimes, Enron, dll). Setiap dataset memiliki dua file utama: file *vocabulary* (vocab*.txt) yang berisi daftar kata unik, dan file *docword* (docword*.txt) yang menyimpan informasi jumlah dokumen (D), jumlah kata unik (W), total kata (N), serta frekuensi kemunculan setiap kata di dalam dokumen tertentu dengan format (*docID*, *wordID*, *count*). Permasalahan utama yang harus diselesaikan dalam proyek *Big Challenge* ini adalah bagaimana memproses dan mengakumulasi total frekuensi kemunculan sebuah kata (yang mungkin tersebar di banyak dokumen / *docID* yang berbeda) ke dalam sebuah struktur data yang efisien di dalam memori utama menggunakan bahasa pemrograman C. Setelah frekuensi setiap kata diakumulasi, program dituntut untuk mampu mencari dan menentukan k kata yang paling sering muncul dalam koleksi dataset tersebut ($10 < k < 150$). Untuk menyelesaikan hal ini, data yang berjumlah hingga ratusan ribu kata tersebut harus diurutkan secara menurun (*descending*) berdasarkan frekuensi kemunculannya. Program juga diwajibkan untuk mengimplementasikan dan membandingkan kinerja waktu komputasi dari tiga algoritma pengurutan yang berbeda, yaitu satu algoritma sederhana (*Insertion Sort*) dan dua algoritma tingkat lanjut (*Quick Sort* dan *Heap Sort*). Hasil akhir pengurutan beserta waktu eksekusinya kemudian harus dapat ditampilkan ke antarmuka layar monitor maupun disimpan secara dinamis ke dalam sebuah file teks.

BAB II

METODE STRUKTUR DATA DAN ALGORITMA

2.1 Desain Program

Program ini dirancang dalam bahasa C dengan pendekatan modular menggunakan paradigma pemrograman terstruktur. Sistem ini bertujuan untuk memproses dataset *Bag of Words* (terdiri dari file kamus kata *vocab*.txt* dan file frekuensi kemunculan kata *docword*.txt*) untuk kemudian diakumulasikan, diurutkan, serta dianalisis performanya. Program ini dipecah menjadi beberapa komponen utama, yakni:

2.1.1 File main.c

Berperan sebagai pengendali utama (*driver*), menyediakan antarmuka menu berbasis teks (*Command Line Interface*) bagi pengguna, menghitung durasi eksekusi pengurutan menggunakan fungsi *clock()*, dan mengarahkan aliran data.

2.1.2 File bagOfWord.h

Berisi definisi konstanta batas memori global dan struktur data dasar (*Data Record*) yang digunakan di seluruh sistem.

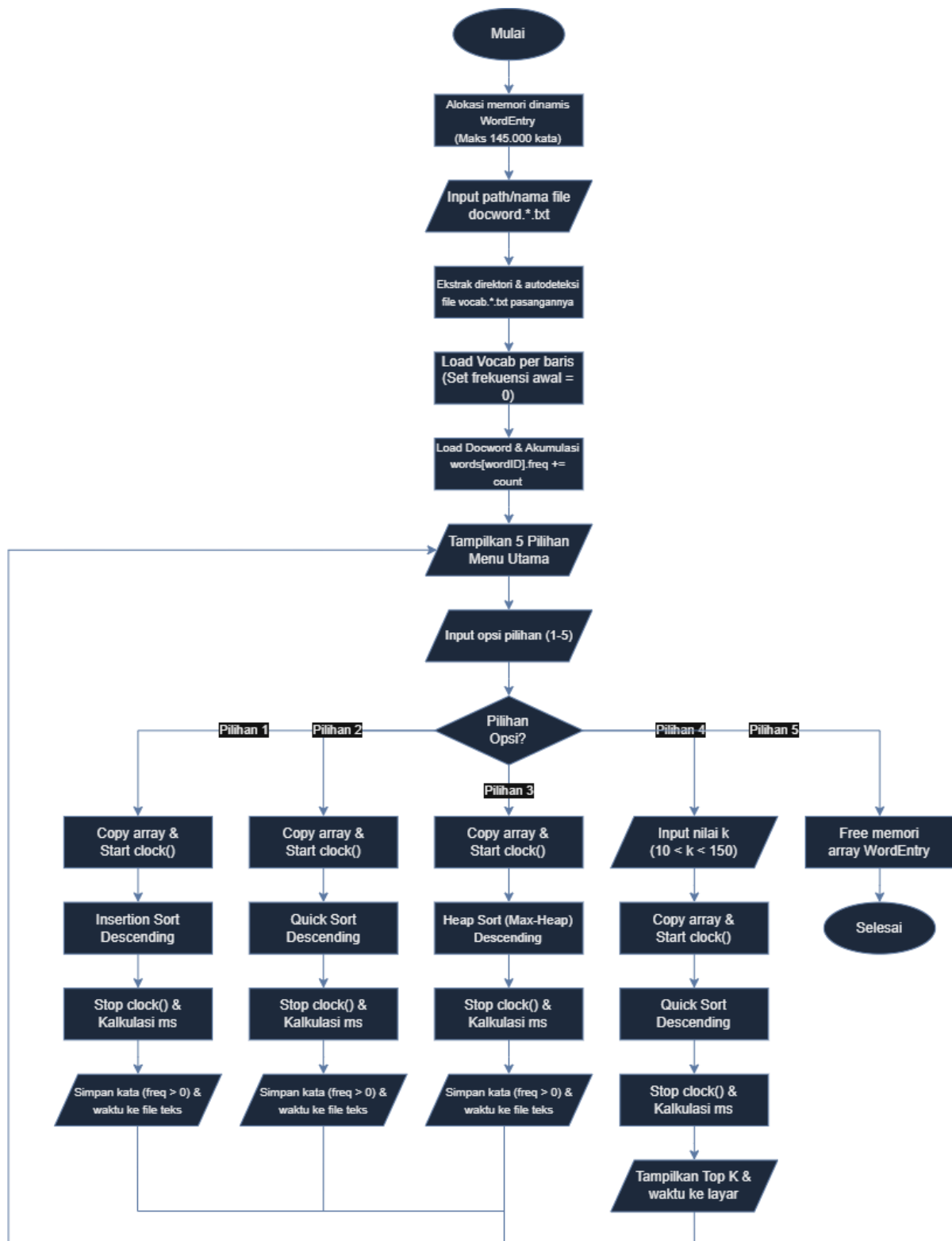
2.1.3 File sorting.c dan sorting.h

Mengimplementasikan tiga algoritma pengurutan berbeda (*Insertion Sort*, *Quick Sort*, *Heap Sort*) yang telah dimodifikasi secara khusus untuk mengurutkan data bertipe struk berdasarkan bobot frekuensi secara menurun (*descending*).

2.1.4 File utils.c dan utils.h

Menyediakan fungsi-fungsi pembantu (*utility*) yang menangani operasi I/O, seperti pemindaian (*parsing*) file teks, duplikasi memori array, pencetakan statistik *Top-K* ke layar, hingga penulisan hasil akhir ke media penyimpanan.

2.2 Flowchart Program



2.3 Struktur Data

Pada program ini kami mendefinisikan dua struktur data utama dalam file bagOfWord.h. Peneraapan ini dilakukan untuk memastikan pengorganisasian memori dilakukan secara efisien

2.3.1 Struktur WordEntry

Digunakan untuk merepresentasikan entitas kata tunggal beserta properti statistiknya setelah diakumulasikan.

```
typedef struct
{
    int wordID;
    char word[64];
    long freq;
} WordEntry;
```

Pada struktur ini, kita akan menggunakannya untuk menyimpan *wordID* (dari *docword*.txt*), kata-kata serta frekuensi kemunculan yang ada pada dataset *vocab*.txt*.

2.3.2 Struktur DatasetHeader

Digunakan untuk menyimpan informasi metadata yang terletak pada tiga baris pertama di dalam file *docword*.txt*

```
typedef struct {
    int D;
    int W;
    long N;
} DatasetHeader;
```

Struktur ini menyimpan data yakni D (total jumlah dokumen unik), W (total kata unik), dan N (total keseluruhan kemunculan kata di seluruh dataset)

2.4 Proses Parsing dan Pembacaan Data

Pada proses pembacaan data dilakukan secara sekuensial pada file *utils.c* untuk menghindari *buffer overflow* serta proses parsing dilakukan untuk mempermudah menampilkan dan mengelola datanya Untuk menentukan seberapa banyak data yang ingin disimpan, kami menggunakan patokan banyaknya data yang ada pada dataset *pubmed* (karena merupakan dataset dengan file paling besar).

```
PIJII at ...\\Big challenge\\punyaGwe via 24.13.0
Get-Content docword.pubmed.txt -Head 5
8200000
141043
483450157
1 6811 1
1 7106 1
```

Dapat dilihat pada gambar bahwa ada 141043 kata unik yang ada pada dataset ini, sehingga dalam strukturnya kami menentukan nilai maksimal untuk menyimpan data adalah 145000 (dapat dilihat di *bagOfWord.h*).

Lalu untuk memuat dataset *vocab*.txt*, kami melakukan parsing terhadap newline dan menggantinya dengan null byte untuk mempermudah menampilkannya nanti, seperti yang terlihat dibawah ini:

```

int load_vocab(char *vocab_file, WordEntry *words, int max_word){
    FILE *fpVocab = fopen(vocab_file, "r");
    if(!fpVocab){
        printf("[!] Gagal memuat file"); return -1;
    }

    int count = 0;
    char line[MAX_WORD_LEN];

    while (fgets(line, sizeof(line), fpVocab) && count < MAX_WORD){
        line[strcspn(line, "\n")] = '\0';

        int id = count + 1;
        words[id].wordID = id;
        strncpy(words[id].word, line, MAX_WORD_LEN-1);
        words[id].word[MAX_WORD_LEN-1] = '\0';
        words[id].freq = 0;

        count++;
    }
    fclose(fpVocab);
    // printf("[DEBUG] Total Vocab = %d\n", count);
    return count;
}

```

Selanjutnya untuk memuat data docword*.txt, kami juga melakukan parsing secara manual untuk membaca file nya karena docword.pubmed.txt termasuk file yang sangat besar jika dibaca menggunakan fungsi dari library c seperti fscanf()

```

DatasetHeader load_docword(char *docword_file, WordEntry *words, int
max_word){
    DatasetHeader header = {0, 0, 0};
    int docID, wordID;
    long countWord = 0;

    FILE *fpDocword = fopen(docword_file, "r");
    if (!fpDocword) {
        printf("[!] Gagal memuat file"); return header;
    }

    if (fscanf(fpDocword, "%d", &header.D) != 1 ||
        fscanf(fpDocword, "%d", &header.W) != 1 ||
        fscanf(fpDocword, "%ld", &header.N) != 1) {
        fprintf(stderr, "ERROR: Format header file docword tidak valid.\n");
        fclose(fpDocword);
        return header;
    }

    // printf("[DEBUG] Header dataset => D=%d W=%d N=%ld\n", header.D,
header.W, header.N);

    // baca per baris data

```



```

char line[64];
while (fgets(line, sizeof(line), fpDocword))
{
    char *ptr = line;

    // Parse docID
    while (*ptr == ' ') {ptr++;}
    docID = 0;
    while(*ptr >= '0' && *ptr <= '9'){
        docID = docID * 10 + (*ptr++ - '0');
    }

    // Parse wordID
    while (*ptr == ' ') {ptr++;}
    wordID = 0;
    while(*ptr >= '0' && *ptr <= '9'){
        wordID = wordID * 10 + (*ptr++ - '0');
    }

    // Parse countWord
    while(*ptr == ' ') {ptr++;}
    countWord = 0;
    while(*ptr >= '0' && *ptr <= '9'){
        countWord = countWord * 10 + (*ptr++ - '0');
    }

    if (wordID >= 1 && wordID <= header.W)
        words[wordID].freq += countWord;
}

fclose(fpDocword);
// printf("[DEBUG] selesai docword\n");
return header;
}

```

Disini kami melakukan parsing untuk membaca data docID, wordID serta frekuensi kata tersebut muncul dalam satu dokumen, dengan melewati spasi lalu mengambil angka sesudah spasi tersebut. Proses ini lebih cepat dibanding memakai fungsi bawaan library c, karena tidak melakukan error checking dan lain sebagainya.

2.5 Analisis Algoritma

2.5.1 Insertion Sort

Pada instruksi diminta untuk melakukan salah satu pengurutan sederhana, disini kami memilih menggunakan Insertion Sort karena secara umum lebih cepat dari Bubble Sort dan Selection Sort karena lebih sedikit melakukan perbandingan dan pertukaran, terutama pada data yang hampir terurut. Algoritma ini bekerja dengan cara

membandingkan dua elemen berdekatan dan menukar jika tidak urut. Proses diulang sebanyak $n-1$. Jika data sudah terurut (kondisi *best-case*), kecepatannya menjadi $O(N)$ karena hanya perlu membandingkan elemen dengan tetangganya.

Best Case	Worst Case	Average Case	Space
$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$

2.5.2 Quick Sort

Algoritma *Quick Sort* diimplementasikan menggunakan metode *Divide and Conquer*, seperti pada fungsi quickSort di file sorting.c, program membagi array menjadi dua sub-array berdasarkan elemen *pivot*. Dalam implementasi ini, kami menggunakan elemen paling akhir dari rentang yang sedang diproses sebagai *pivot* (long pivot = arr[high].freq;).

Proses partisi dimodifikasi secara khusus agar mengurutkan data secara *descending*, sehingga program menelusuri array dari indeks terendah hingga sebelum *pivot*. Jika ditemukan elemen dengan nilai frekuensi yang lebih besar atau sama dengan *pivot* (arr[j].freq >= pivot), elemen tersebut akan ditukar (swap) ke bagian kiri array. Setelah seluruh elemen dibandingkan, *pivot* diletakkan pada posisi tengah yang tepat, dan fungsi quickSort dipanggil kembali secara rekursif untuk sub-array bagian kiri dan kanan.

Best Case	Worst Case	Average Case	Space
$O(N \log N)$	$O(N^2)$	$O(N \log N)$	$O(\log N)$

Quick Sort memberikan performa rata-rata yang sangat cepat ($O(N \log N)$) karena proses partisi dilakukan secara *in-place* tanpa membutuhkan alokasi memori array tambahan. *Worst case* ($O(N^2)$) secara teoritis dapat terjadi jika data sudah terurut sempurna secara terbalik, namun dalam kasus distribusi kata pada dataset teks natural seperti *Bag of Words*, performa yang dihasilkan sangat optimal dan waktu eksekusinya jauh melampaui *Insertion Sort*.

2.5.3 Heap Sort

Berbasis pada struktur data pohon biner lengkap (*Complete Binary Tree*). Program pertama-tama membangun sebuah *Max-Heap* melalui fungsi heapify() dari bawah ke atas. Karena struktur ini mengharuskan nilai induk selalu lebih besar dari anaknya, elemen akar (indeks 0) dipastikan memiliki nilai frekuensi terbesar. Elemen akar ini kemudian ditukar dengan elemen paling akhir dari heap, ukuran heap

diperkecil, dan proses `heapify()` diulang kembali pada akar baru hingga heap habis. Di akhir proses, program membalik urutan array agar menjadi *descending*.

Best Case	Worst Case	Average Case	Space
$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(1)$

Pada setiap *case* nya Heap Sort sangat konsisten karena pada saat proses *heapify*, prosesnya paling mentok hanya menelusuri jalur sedalam tingkat pohonnya yang hanya $O(\log N)$, lalu pada pengurutannya program melakukan *looping* sebanyak $n-1$ untuk mengeluarkan elemen terbesar dari akar pohon, sehingga total eksekusinya menjadi $(N - 1) * \log N$ atau setara dengan $N \log N$. Dengan proses ini Heap sort memaksakan struktur data array diorganisasikan ke dalam pohon biner yang seimbang dan performa komputasi nya berjalan stabil.

2.6 Penyimpanan hasil ke dalam file text

Untuk memenuhi spesifikasi tugas yang mewajibkan penyimpanan hasil keluaran, program menggunakan fungsi `saveToFile` yang didefinisikan di dalam `utils.c`. Fungsi ini menerima empat parameter utama: nama file keluaran yang digenerasi secara dinamis (contoh: `output_quicksort_pubmed.txt`), *pointer* ke array hasil pengurutan, jumlah kosakata (N), dan nilai waktu komputasi (*time_taken*).

Di dalam fungsinya, I/O dijalankan dengan membuka file pada mode *write* ("w"). Algoritma kemudian melakukan iterasi terhadap seluruh isi array. Untuk mengoptimalkan ukuran file keluaran dan mencegah data sampah terbawa, program menerapkan filter kondisi `if (arr[i].freq > 0)`. Hal ini memastikan bahwa hanya kosakata yang benar-benar muncul dalam dokumen yang akan dicetak ke dalam file menggunakan fungsi `fprintf` dengan format kata (frekuensi).

```
for (int i = 0; i < n; i++) {
    if (arr[i].freq > 0) {
        fprintf(fp, "%s (%ld)\n", arr[i].word, arr[i].freq);
    }
}
fprintf(fp, "\nWaktu untuk mengurutkan: %.0f ms\n", time_taken);
```

Setelah semua kata dituliskan ke dalam file, program mencetak rekapitulasi waktu eksekusi pengurutan di baris paling bawah. Memori *buffer* penulisan kemudian ditutup menggunakan `fclose(fp)`, memastikan keamanan data di dalam memori utama terbebas dari *memory leak*.

BAB III

HASIL PROGRAM

3.1 Tampilan Awal dan Pemuatan Dataset

Saat eksekusi awal, program akan meminta pengguna untuk memasukkan direktori dan nama file dari dataset utama (contoh: docword.pubmed.txt). Sistem akan memberikan indikator bahwa kedua file (docword dan vocab) sedang dimuat ke dalam memori struktur data.

```
> mingw32-make
gcc -Wall -pedantic -g main.c utils.c sorting.c -o bigchallenge
> ./bigchallenge
Tentukan file docword: docword.pubmed.txt
```

3.2 Tampilan Menu Utama

Setelah proses akumulasi data di memori selesai dilakukan, layar interaktif akan menampilkan menu 5 pilihan spesifik sesuai dengan instruksi fungsional dari tugas *Big Challenge*.

```
> ./bigchallenge
Tentukan file docword: docword.pubmed.txt

[i] Sedang memuat vocab.pubmed.txt dan docword.pubmed.txt...

Pilihan:
  1) Urutkan dengan Insertion Sort   (simpan ke file)
  2) Urutkan dengan Quick Sort       (simpan ke file)
  3) Urutkan dengan Heap Sort        (simpan ke file)
  4) Tampilkan k kata terbesar ke layar
  5) Selesai
Pilihan anda: 
```

3.3 Output Proses Pengurutan dan Pembuatan File

Gambar di bawah ini merupakan respon program saat pengguna memilih metode pengurutan. Program akan menjalankan komputasi *sorting*, menampilkan lamanya waktu yang dihabiskan untuk mengurutkan data secara *descending*, dan memberikan notifikasi keberhasilan pencetakan data ke dalam file .txt.

3.3.1 Menu pilihan pertama

```
Pilihan:
  1) Urutkan dengan Insertion Sort   (simpan ke file)
  2) Urutkan dengan Quick Sort       (simpan ke file)
  3) Urutkan dengan Heap Sort        (simpan ke file)
  4) Tampilkan k kata terbesar ke layar
  5) Selesai
Pilihan anda: 1

[i] Memulai Insertion Sort...

Selesai mengurutkan, Waktu yang diperlukan: 113896 ms
[i] Keluaran berhasil disimpan ke dalam file: output_insertion_pubmed.txt
```

3.3.2 Menu pilihan kedua

```
Pilihan:
1) Urutkan dengan Insertion Sort (simpan ke file)
2) Urutkan dengan Quick Sort (simpan ke file)
3) Urutkan dengan Heap Sort (simpan ke file)
4) Tampilkan k kata terbesar ke layar
5) Selesai
Pilihan anda: 2

[i] Memulai Quicksort...

Selesai mengurutkan, Waktu yang diperlukan: 454 ms
[i] Keluaran berhasil disimpan ke dalam file: output_quicksort_pubmed.txt
```

3.3.3 Menu pilihan ketiga

```
Pilihan:
1) Urutkan dengan Insertion Sort (simpan ke file)
2) Urutkan dengan Quick Sort (simpan ke file)
3) Urutkan dengan Heap Sort (simpan ke file)
4) Tampilkan k kata terbesar ke layar
5) Selesai
Pilihan anda: 3

[i] Memulai Heapsort...

Selesai mengurutkan, Waktu yang diperlukan: 87 ms
[i] Keluaran berhasil disimpan ke dalam file: output_heapsort_pubmed.txt
```

3.4 Isi File Output

Berikut adalah tangkapan layar dari file teks *output* yang dihasilkan secara otomatis oleh sistem. File tersebut berisi daftar kosakata yang paling sering muncul, diurutkan dari yang terbesar, beserta rekapitulasi waktu di baris paling bawah.

3.4.1 File output Insertion Sort

```
≡ output_insertion_pubmed.txt U X C utils.
≡ output_insertion_pubmed.txt
1 patient (7499755)
2 cell (4071002)
3 effect (3429503)
4 protein (2473491)
5 treatment (2411160)
6 level (2309211)
7 activity (1959728)
8 human (1739762)
9 rat (1723064)
10 concentration (1561419)

141041 caec (88)
141042 hib5 (86)
141043 lict (82)
141044
141045 Waktu untuk mengurutkan: 113896 ms
141046
```

3.4.2 File output Quick Sort

```
output_quicksort_pubmed.txt U X  output_
output_quicksort_pubmed.txt
1  patient (7499755)
2  cell (4071002)
3  effect (3429503)
4  protein (2473491)
5  treatment (2411160)
6  level (2309211)
7  activity (1959728)
8  human (1739762)
9  rat (1723064)
10 concentration (1561419)

141041 caec (88)
141042 hib5 (86)
141043 lict (82)
141044
141045 Waktu untuk mengurutkan: 454 ms
141046
```

3.4.3 File output Heap Sort

```
output_heapsort_pubmed.txt U X  output_
output_heapsort_pubmed.txt
1  patient (7499755)
2  cell (4071002)
3  effect (3429503)
4  protein (2473491)
5  treatment (2411160)
6  level (2309211)
7  activity (1959728)
8  human (1739762)
9  rat (1723064)
10 concentration (1561419)

141041 caec (88)
141042 hib5 (86)
141043 lict (82)
141044
141045 Waktu untuk mengurutkan: 87 ms
141046
```

3.4 Output K Kata Teratas di Layar Monitor

Ketika pengguna memilih menu ke-4, program akan meminta besaran nilai k antara 10 hingga 150. Program kemudian mencetak secara rapi tabel yang berisi Peringkat (No), Kata, dan Frekuensinya. Eksekusi ini diakhiri dengan tampilan waktu yang dibutuhkan untuk menyelesaikan proses pencarian top k tersebut.

```

Pilihan:
1) Urutkan dengan Insertion Sort    (simpan ke file)
2) Urutkan dengan Quick Sort        (simpan ke file)
3) Urutkan dengan Heap Sort         (simpan ke file)
4) Tampilkan k kata terbesar ke layar
5) Selesai
Pilihan anda: 4

Masukkan nilai k (10 < k < 150): 15

[i] Mengurutkan data menggunakan Quicksort untuk mendapatkan Top 15...

No.      Kata                      Frekuensi
-----
1      patient                    7499755
2      cell                      4071002
3      effect                    3429503
4      protein                   2473491
5      treatment                 2411160
6      level                    2309211
7      activity                 1959728
8      human                    1739762
9      rat                      1723064
10     concentration            1561419
11     data                    1440714
12     control                 1378408
13     system                  1323283
14     analysis                1273676
15     receptor                1266427

Waktu untuk mengurutkan: 443 ms

```

3.5 Pengecekan memory leaks

```

(eruma@kali)-[~/Downloads/BIGCHALLENGE-DSA-main]
$ valgrind ./bigchallenge
==25906== Memcheck, a memory error detector
==25906== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==25906== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==25906== Command: ./bigchallenge
==25906==
Tentukan file docword: docword.pubmed.txt

[i] Sedang memuat vocab.pubmed.txt dan docword.pubmed.txt...

Pilihan:
1) Urutkan dengan Insertion Sort    (simpan ke file)
2) Urutkan dengan Quick Sort        (simpan ke file)
3) Urutkan dengan Heap Sort         (simpan ke file)
4) Tampilkan k kata terbesar ke layar
5) Selesai
Pilihan anda: 5

[i] Program Selesai.
==25906==
==25906== HEAP SUMMARY:
==25906==    in use at exit: 0 bytes in 0 blocks
==25906==   total heap usage: 7 allocs, 7 frees, 11,611,264 bytes allocated
==25906==
==25906== All heap blocks were freed -- no leaks are possible
==25906==
==25906== For lists of detected and suppressed errors, rerun with: -s
==25906== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

(eruma@kali)-[~/Downloads/BIGCHALLENGE-DSA-main]
$

```

BAB IV

KESIMPULAN

Berdasarkan proses perancangan, implementasi *source code*, hingga tahap evaluasi program *Big Challenge* ini, dapat ditarik beberapa kesimpulan utama:

1. Pemilihan Struktur Data yang Efisien Penggunaan *Array of Struct* yang dikombinasikan dengan alokasi memori dinamis (*calloc*) terbukti sangat efektif untuk menampung ratusan ribu *vocabulary*. Struktur ini memungkinkan program memetakan *wordID* ke nama kata (*string*) dan mengakumulasikan frekuensinya secara logis (dengan kompleksitas pencarian index $O(1)$) saat mem-*parsing* file *docword* dalam jumlah baris yang masif.
2. Efisiensi Algoritma Pengurutan Berdasarkan Skala Data Kinerja algoritma pengurutan sangat bergantung pada kompleksitas waktu teoritisnya. *Insertion Sort*, dengan kompleksitas waktu $O(N^2)$, terbukti tidak efisien dan rentan terhadap *bottleneck* ketika diterapkan pada dataset besar. Sebaliknya, pendekatan *Divide and Conquer* pada *Quick Sort* dan optimasi *Complete Binary Tree* pada *Heap Sort* yang sama-sama memiliki waktu rata-rata $O(N \log N)$ berhasil mengurutkan dataset berskala masif secara instan dalam hitungan milidetik.
3. Keandalan Proses Input/Output (I/O) Pemanfaatan fungsi *fscanf* dalam pembacaan file terbukti lebih andal (*robust*) karena kemampuannya mengabaikan berbagai karakter tidak kasatmata (seperti kelebihan spasi, tabulasi, hingga karakter *newline* ganda bawaan sistem operasi) dibandingkan pemrosesan iterasi manual. Program juga berhasil melakukan proses tulis (*write*) ke dalam file eksternal untuk menyimpan luaran data yang telah difilter tanpa memicu indikasi kebocoran memori (*memory leak*).

Secara keseluruhan, proyek ini membuktikan bahwa efisiensi perangkat lunak berskala besar tidak bisa dicapai hanya melalui kecepatan komputasi peranti keras (*hardware*), melainkan sangat bergantung pada pemilihan struktur data yang efisien dan implementasi algoritma tingkat lanjut yang tepat.

LAMPIRAN

Repository Project Big-Challenge: <https://github.com/Eruumaa/BIGCHALLENGE-DSA>

Link Dataset: <https://archive.ics.uci.edu/dataset/164/bag+of+words>

Link Video Penjelasan: <https://youtu.be/xXKTgXLwh-s>